

FOREX Arbitrage Seeker GUI

with

ccxt package

Inhaltsverzeichnis

1. Project description.....	3
Aim.....	3
🎯 Primary Aim.....	3
⚙️ Functional Goals.....	3
🧠 Why It Matters.....	4
2. Planning section.....	5
3. Pythonic implementation of the FOREX Arbitrage Seeker GUI.....	7
4. Arbitrage seeking GUI structure.....	12
5. Future improvements.....	17
6. References.....	19

1. Project description

Aim

The purpose of a FOREX Arbitrage Seeker GUI is to act as a modular, real-time analytical tool that enables users—especially those with quantitative or technical backgrounds—to identify, visualize, and simulate arbitrage opportunities across cryptocurrency markets.

Primary Aim

To provide a **hands-on decision-support system** that helps users:

- Detect profitable arbitrage windows (spatial or triangular)
- Analyze real-time spreads between market pairs
- Simulate trade outcomes based on live pricing
- Visualize trade paths interactively
- Log and export arbitrage events for analysis or backtesting

Functional Goals

- **Market Monitoring:** Continuously track live prices (bid/ask) from Binance and compute spreads on user-selected pairs.
- **Simulation:** Allow users to test hypothetical trades and view potential returns before acting.
- **Triangular Arbitrage:** Identify and evaluate profitable circular conversion paths such as $\text{USDT} \rightarrow \text{BTC} \rightarrow \text{ETH} \rightarrow \text{USDT}$.
- **Interactive Visualization:** Present triangle loops visually using `QGraphicsScene`, making abstract relationships tangible.
- **Analytics & Logging:** Log each opportunity, generate spread charts, and support export for offline analysis.
- **User Control:** Offer manual refresh, auto-refresh toggling, and selective export of arbitrage findings.

Why It Matters

In fast-moving crypto markets, arbitrage opportunities are fleeting. This tool arms users with:

- Clarity: Real-time insight into pricing anomalies
- Speed: Auto-refreshing mechanics and instant simulation
- Intuition: Graphical triangle mapping for quick interpretation
- Reproducibility: Logged data that can feed back into research or trading models

2. Planning section

Real-time arbitrage opportunities are both a data engineering challenge *and* a UI design challenge. We are blending finance, systems programming, and UX. Here's how one could approach it, step-by-step:

1. Core Architecture Overview

Think modular:

```
[ Market Data Collector ] → [ Arbitrage Engine ] → [ Logging/Simulation ] → [ GUI Display ]
```

2. Data Feeds & APIs

- Choose 2–5 exchanges (start with public APIs from Binance, Coinbase, Kraken, Bitfinex, etc.).
- Use WebSockets (if supported) for real-time price streams — much faster than polling REST endpoints.
- Normalize tickers (e.g. BTC-USD vs. BTC/USD vs. XBT-USD).

 Suggested libs:

- `websockets`, `aiohttp`, or `requests`
- For crypto: `ccxt` (supports many exchanges out of the box)

3. Arbitrage Engine

- For each common trading pair (e.g. BTC/USD), identify price deltas:
 - **Buy low** on one exchange → **Sell high** on another
- Factor in:
 - Fees (taker/maker fees)
 - Slippage (maybe add a buffer)
 - Network transfer time (huge for on-chain arbitrage)

 Start with a simple condition:

```
if price_binance < price_coinbase * (1 - fee_buffer):  
    log_opportunity(...)
```

4. Logging & Simulation

- Store arbitrage events with timestamp, spread %, and simulated trade volume
- Enable "Replay Mode" to analyze missed chances
- Consider profit simulator: apply balance constraints + transfer delays

 Suggested tools:

- `pandas` , `SQLite` , or lightweight in-memory structures

5. GUI Design

- Use `PyQt5` , `Tkinter` , or even `Streamlit` for quick prototyping
- Live table of arbitrage signals (sortable by spread, age)
- Charts: price convergence/divergence over time
- Pause/play simulation button

6. Extra Credit (for later)

- Email or Telegram alerts
- Add fiat/crypto conversion paths (e.g. ETH-BTC-USD triangle)
- Live arbitrage heat map
- Auto-trading simulation (Paper bot mode)

🕒 GUI Layout Sketch (Textual Wireframe)

⚡ Arbitrage Opportunity Scanner					
[Select Exchanges] [BTC/USD] [ETH/USDT] [🔄 Refresh]					
Exchange	Pair	Bid Price	Ask Price	Spread %	

Binance	BTC/USD	\$29,503	\$29,508	0.017%	
Coinbase	BTC/USD	\$29,512	\$29,520	0.027%	
Kraken	BTC/USD	\$29,498	\$29,510	0.041%	

[Simulate Trade] [Export Logs]					
⌚ Arbitrage Log Console					
[00:01] 🟢 Opportunity Detected (Binance → Kraken: +0.04%)					
[00:07] 🔄 No spread over 0.1%					

3. Pythonic implementation of the FOREX Arbitrage Seeker GUI

Here is a complete, self-contained Python script that combines all key features of our Arbitrage Seeker GUI so far:

- Real-time price fetching via `ccxt`
- Table display with spread analysis
- Auto-refresh toggle
- Arbitrage log console
- Trade simulation dialog
- Triangle arbitrage detection
- Visual triangle rendering with `QGraphicsScene`
- Export log to CSV

⚠ Requirements:

- `!pip install pyqt5 ccxt matplotlib`
- `!pip install streamlit pandas numpy plotly requests websocket-client`

```

import sys, datetime
from PyQt5.QtWidgets import (
    QApplication, QWidget, QVBoxLayout, QTableWidget, QTableWidgetItem,
    QPushButton, QComboBox, QLabel, QTextEdit, QHBoxLayout,
    QLineEdit, QDialog, QDialogButtonBox, QFormLayout, QMessageBox,
    QGraphicsScene, QGraphicsView, QGraphicsTextItem
)
from PyQt5.QtGui import QPen, QBrush, QFont
from PyQt5.QtCore import Qt, QTimer, QPointF
import ccxt
from matplotlib.backends.backend_qt5agg import FigureCanvasQTagg as FigureCanvas
from matplotlib.figure import Figure

class SimulateDialog(QDialog):
    def __init__(self, buy_price, sell_price, parent=None):
        super().__init__(parent)
        self.setWindowTitle("🔍 Simulate Arbitrage Trade")
        layout = QFormLayout()

        self.volume_input = QLineEdit()
        self.volume_input.setPlaceholderText("e.g. 1000 (USDT)")
        layout.addRow("Trade Volume:", self.volume_input)

        self.buy_price = buy_price
        self.sell_price = sell_price

        buttons = QDialogButtonBox(QDialogButtonBox.Ok | QDialogButtonBox.Cancel)
        buttons.accepted.connect(self.simulate)
        buttons.rejected.connect(self.reject)
        layout.addWidget(buttons)
        self.setLayout(layout)

    def simulate(self):
        try:
            volume = float(self.volume_input.text())
            units = volume / self.buy_price
            proceeds = units * self.sell_price
            profit = proceeds - volume
            QMessageBox.information(self, "Simulated Profit", f"📈 Profit: {profit:.2f}\n📉 Net Spread: {(self.sell_price - sel
            self.accept()
        except Exception as e:
            QMessageBox.warning(self, "Error", f"⚠️ Invalid input: {e}")

```

```

class ArbitrageScanner(QWidget):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("⚡ Arbitrage Seeker")
        self.setGeometry(100, 100, 950, 700)
        layout = QVBoxLayout()

        # Header
        header = QHBoxLayout()
        self.symbol_selector = QComboBox()
        self.symbol_selector.addItem("BTC/USDT")
        self.symbol_selector.addItem("ETH/USDT")
        refresh_btn = QPushButton("🔄 Refresh")
        refresh_btn.clicked.connect(self.fetch_prices)
        header.addWidget(QLabel("Pair:"))
        header.addWidget(self.symbol_selector)
        header.addWidget(refresh_btn)

        # Table
        self.table = QTableWidgetItem(1, 5)
        self.table.setHorizontalHeaderLabels(["Exchange", "Pair", "Bid", "Ask", "Spread %"])

        # Buttons
        btns = QHBoxLayout()
        self.pause_btn = QPushButton("⏸ Pause Auto-Refresh")
        self.pause_btn.setCheckable(True)
        self.pause_btn.clicked.connect(self.toggle_refresh)
        sim_btn = QPushButton("Simulate Trade")
        sim_btn.clicked.connect(self.simulate_trade)
        tri_btn = QPushButton("▲ Triangle Scan")
        tri_btn.clicked.connect(self.triangle_arbitrage)
        export_btn = QPushButton("Export Logs")
        export_btn.clicked.connect(self.export_log)
        btns.addWidget(sim_btn)
        btns.addWidget(tri_btn)
        btns.addWidget(export_btn)
        btns.addWidget(self.pause_btn)

        # Log
        self.log = QTextEdit()
        self.log.setReadOnly(True)
        self.log.append("🟢 System initialized.")

        # Chart
        self.fig = Figure(figsize=(5,2))
        self.canvas = FigureCanvas(self.fig)
        self.spread_history = []

```

```

# Diagram
self.scene = QGraphicsScene()
self.view = QGraphicsView(self.scene)
self.view.setMinimumHeight(250)

layout.addLayout(header)
layout.addWidget(self.table)
layout.addLayout(btns)
layout.addWidget(QLabel("📊 Spread History"))
layout.addWidget(self.canvas)
layout.addWidget(QLabel("📐 Triangle Arbitrage Diagram"))
layout.addWidget(self.view)
layout.addWidget(QLabel("📝 Arbitrage Log"))
layout.addWidget(self.log)
self.setLayout(layout)

self.ex = ccxt.binance()
self.timer = QTimer(self)
self.timer.setInterval(5000)
self.timer.timeout.connect(self.fetch_prices)
self.timer.start()
self.fetch_prices()

def fetch_prices(self):
    symbol = self.symbol_selector.currentText()
    try:
        ticker = self.ex.fetch_ticker(symbol)
        bid, ask = ticker['bid'], ticker['ask']
        spread = round(((ask - bid) / bid) * 100, 4)
        self.table.setItem(0, 0, QTableWidgetItem("Binance"))
        self.table.setItem(0, 1, QTableWidgetItem(symbol))
        self.table.setItem(0, 2, QTableWidgetItem(f"{bid:.2f}"))
        self.table.setItem(0, 3, QTableWidgetItem(f"{ask:.2f}"))
        self.table.setItem(0, 4, QTableWidgetItem(f"{spread:.3f}%"))

        self.spread_history.append(spread)
        if len(self.spread_history) > 20: self.spread_history.pop(0)
        self.plot_spread()
    except Exception as e:
        self.log.append(f"⚠️ Fetch failed: {e}")

```

```

def toggle_refresh(self):
    if self.pause_btn.isChecked():
        self.timer.stop()
        self.pause_btn.setText("▶ Resume")
        self.log.append("⏸ Paused.")
    else:
        self.timer.start()
        self.pause_btn.setText("⏸ Pause Auto-Refresh")
        self.log.append("▶ Resumed.")

def plot_spread(self):
    self.fig.clear()
    ax = self.fig.add_subplot(111)
    ax.plot(self.spread_history, marker='o', color='green')
    ax.set_ylabel("Spread %")
    ax.set_title("Live Spread Tracking")
    self.canvas.draw()

def simulate_trade(self):
    try:
        bid = float(self.table.item(0,2).text())
        ask = float(self.table.item(0,3).text())
        dlg = SimulateDialog(ask, bid, self)
        dlg.exec_()
    except:
        self.log.append("⚠ Simulation failed.")

def export_log(self):
    fname = f"arb_log_{datetime.datetime.now().strftime('%H%M%S')}.csv"
    with open(fname, "w") as f:
        for line in self.log.toPlainText().splitlines():
            f.write(line + "\n")
    self.log.append(f"✅ Exported to {fname}")

```

```

def triangle_arbitrage(self):
    try:
        b = self.ex.fetch_ticker("BTC/USD")
        e = self.ex.fetch_ticker("ETH/BTC")
        u = self.ex.fetch_ticker("ETH/USD")

        usd = 1000
        step1 = usd / b["ask"]
        step2 = step1 * e["bid"]
        final = step2 * u["bid"]
        profit = final - usd
        spread = (profit / usd) * 100
        result = f"🔄 Route: USD → BTC → ETH → USD\n💰 Profit: {profit:.2f} ({spread:.3f}%)"
        self.log.append(result)
        self.draw_triangle(["USD", "BTC", "ETH"], [f"1/{b['ask']:.0f}", f"{e['bid']:.3f}", f"{u['bid']:.0f}"], spread)
    except Exception as e:
        self.log.append(f"⚠️ Triangle check failed: {e}")

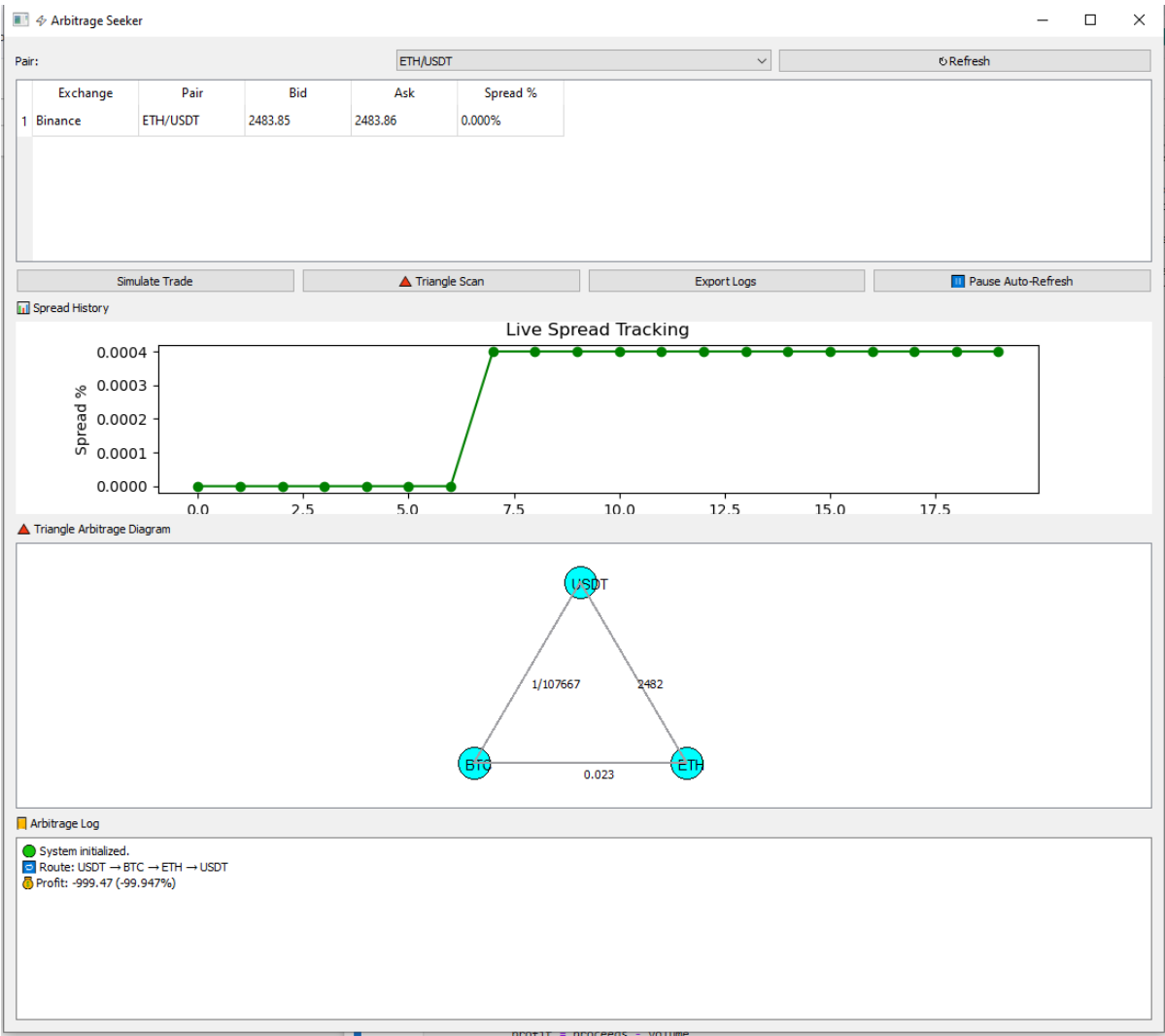
def draw_triangle(self, labels, values, spread):
    self.scene.clear()
    pos = [QPointF(150, 30), QPointF(50, 200), QPointF(250, 200)]
    for i, label in enumerate(labels):
        circle = self.scene.addEllipse(pos[i].x()-15, pos[i].y()-15, 30, 30,
                                       pen=QPen(Qt.black), brush=QBrush(Qt.cyan))

        txt = QGraphicsTextItem(label)
        txt.setFont(QFont("Arial", 10))
        txt.setPos(pos[i].x()-12, pos[i].y()-10)
        self.scene.addItem(txt)
    for i in range(3):
        start, end = pos[i], pos[(i+1)%3]
        color = Qt.green if spread > 0 else Qt.gray
        pen = QPen(color, 2)
        self.scene.addLine(start.x(), start.y(), end.x(), end.y(), pen)
        mid = (start + end) / 2
        lbl = QGraphicsTextItem(values[i])
        lbl.setPos(mid.x(), mid.y())
        self.scene.addItem(lbl)

if __name__ == "__main__":
    app = QApplication(sys.argv)
    window = ArbitrageScanner()
    window.show()
    sys.exit(app.exec_())

```

4. Arbitrage seeking GUI structure



🧠 Arbitrage Seeker GUI: Technical & Functional Analysis

The **Arbitrage Seeker** is a full-featured GUI application designed to detect crypto arbitrage opportunities on Binance, including traditional spread analysis, simulated trade evaluation, and triangle arbitrage visualization. Below is a structured deep-dive of its architecture, powered by `PyQt5`, `ccxt`, and `matplotlib`.

🖼️ Interface Overview (Referencing Uploaded Image)

The GUI layout is composed of the following layers from top to bottom:

1. Pair Selector and Manual Refresh
2. Live Price Table
3. Control Panel
4. Live Spread History Chart
5. Triangle Arbitrage Visualizer
6. Arbitrage Event Log

Each component is interactive or self-updating, contributing to the analytical utility of the interface.

1. Initialization and Layout

The `ArbitrageScanner` class is the core window. It extends `QWidget` and sets up:

- Timer-based auto-refresh (5-second intervals)
- Dropdowns and interactive buttons
- Graph canvas for plotting spreads
- Graphics scene for triangle loops
- A text log terminal for output

2. Price Fetching Logic

Method: `fetch_prices`

- Retrieves live ticker data from Binance via `ccxt`
- Parses bid/ask from the selected pair
- Computes spread:
$$[\text{Spread}] = \frac{\text{Ask} - \text{Bid}}{\text{Bid}} \times 100$$
- Updates the UI table and appends the spread to a list for plotting

Handles exceptions with log messages for fault tolerance.

3. Auto-Refresh Toggle

Method: `toggle_refresh`

- Uses a `QTimer` to enable/disable periodic data pulls
- Button text switches between  Pause and  Resume
- Provides log feedback on state changes

4. Trade Simulation

Class: `SimulateDialog`

Triggered by: `simulate_trade`

- Opens a modal input field for user to simulate USDT-based trades
- Computes:

```
units = volume / buy_price
proceeds = units * sell_price
profit = proceeds - volume
```

- Populates result via a message box
- Handles invalid input gracefully

5. Spread History Visualization

Method: `plot_spread`

- Maintains a ring buffer of up to 20 spread values
- Renders a line graph of recent spreads with `matplotlib`

Purpose: detect volatility patterns or potential widening spreads.

6. Log Exporting

Method: `export_log`

- Dumps all entries from the QTextEdit log terminal to a `.csv` file
- Timestamped filename: `arb_log_HHMMSS.csv`

A lightweight way to archive analytical sessions.

7. Triangle Arbitrage Engine

Method: `triangle_arbitrage`

Simulates:

```
USDT → BTC → ETH → USDT
```

Steps:

- Buy BTC with USDT at `BTC/USDT` ask
- Convert BTC → ETH at `ETH/BTC` bid
- Convert ETH → USDT at `ETH/USDT` bid
- Compute net profit and spread:

```
profit = final - initial  
spread = (profit / initial) * 100
```

Outputs results and passes values to the triangle visualizer.

8. Triangle Visualization

Method: `draw_triangle`

Draws a directional graph with:

- **3 Nodes** (assets)
- **3 Arrows** (trades)
- **Edge Labels** (rates)
- **Arrow Coloring:**
 -  Green: Profitable path
 -  Gray: Non-profitable or neutral

Uses `QGraphicsScene` and `QGraphicsTextItem` for rendering. Layout is hardcoded into a triangular coordinate map.

Technologies Used

Category	Tools / Libraries
UI Framework	PyQt5
Live Data Feed	ccxt (Binance client)
Data Plotting	matplotlib , FigureCanvas
Graphics Engine	QGraphicsScene , QGraphicsView
Simulation	QDialog , QLineEdit , QMessageBox
Log Management	QTextEdit + file export

Notable Design Strengths

- Modular architecture and event-driven flow
- Clean visual demarcation between live data, analytics, and controls
- Separation of concerns: simulation, fetching, drawing
- Room for scaling: triangle logic can be generalized to N-currency loops
- Resilient: uses logging and exception handlers to avoid crashes

Opportunities for Enhancement

- Generalize triangle scanner to explore more currency cycles dynamically
- Add real-time multi-exchange support with spread ranking
- Log viewer with color-coded severity (info, warning, error)
- Configurable fees and slippage in simulation
- Deploy as a standalone executable or web-based frontend via PyWebView

5. Future improvements

Our Arbitrage Seeker GUI is already incredibly capable, but there's plenty of room to sharpen its edge and expand its scope. Here's a curated list of **future enhancements**, grouped by **category** to help prioritize development:

Data & Market Intelligence

- **Multi-Exchange Comparison**
Track bid/ask prices across multiple exchanges (e.g., Coinbase, Kraken) side-by-side, enabling classic arbitrage opportunities.
- **Multi-Asset Scanning**
Automatically loop over many trading pairs (BTC/ETH, ETH/USDT, XRP/BTC, etc.) and flag arbitrage openings in a ranked list.
- **Real-Time Order Book Depth**
Fetch full L2 data to estimate realistic tradeable volume and slippage rather than relying on top-of-book prices.
- **Custom Fee Models & Spread Filters**
Let users input exchange-specific fees and display only opportunities above net profitability thresholds.

Analytics & Visualization

- **Historical Spread Charting**

UX & Interaction

- **Dark Mode Toggle** 
Give your GUI those sleek fintech vibes and reduce glare for late-night arbitrage hunting.
- **Auto-Detect Best Arbitrage**
Highlight the most profitable route in green and auto-select it for simulation or trade.
- **Notification System**
Send alerts via desktop popup, email, or Telegram when a spread exceeds a user-defined threshold.
- **Trade Executor Sandbox** 
Add mock trade execution buttons to simulate realistic end-to-end orders without placing real trades.

Developer Tools & Expandability

- **Pluggable Exchange Framework**
Add/disable exchanges dynamically via a settings panel, using `ccxt`'s unified API.
- **REST API Integration**
Expose key features (log export, triangle scanner, trade simulation) as endpoints for broader integration.
- **Custom Plugin Interface**
Allow users to write their own Python classes to define new arbitrage strategies or visualizations.

Advanced Features (Stretch Goals)

- **Triangular Arbitrage Generalization**
Use graph theory to scan the exchange's full pair graph and automatically uncover all 3-node profitable cycles.
- **Backtesting Engine**
Replay historical spreads and simulate virtual trading outcomes to validate strategy performance.
- **Machine Learning Prediction**
Train models to predict spread widening or estimate how long an arbitrage window might remain open.
- **Auto Trader Mode (Paper Only)**
Build a paper-trading module that places hypothetical trades and tracks virtual balances, simulating portfolio growth.

6. References

1. R. W. Melicher, E. A. Norton: "Introduction to Finance", 16th Ed. WILEY (2017); Anatoly B. Schmidt: "Quantitative Finance for Physicists: An Introduction", 1st Ed. Academic Press (2005); Alex Backwell: "An Intuitive Introduction to Finance and Derivatives: Concepts, Terminology and Models", 1st Ed, Springer (2023); Michael Isichenko: "Quantitative Portfolio Management: The Art and Science of Statistical Arbitrage", 1st Ed., Springer (2021); John H. Cochrane: "Asset Pricing", Revised Ed., Princeton University Press (2005); Antti Ilmanen: "Expected Returns: An Investor's Guide to Harvesting Market Rewards", 1st Ed., WILEY (2011); Steven E. Shreve: "Stochastic Calculus for Finance I & II", 1st Ed., Springer (2004); Andrew Pole: "Statistical Arbitrage: Algorithmic Trading Insights and Techniques", 1st Ed., WILEY (2007); Mark S. Joshi: "The Concepts and Practice of Mathematical Finance", 2nd Ed., Cambridge University Press (2008);
Software: ccxt-documentation: <https://docs.ccxt.com/#/>, <https://ccxtcn.readthedocs.io/zh-cn/latest/> and <https://pypi.org/project/ccxt-download/>.
2. A. Meister , T. Sonar: "Numerik", 1st Ed. Springer-Spektrum (2019); S. Chapra, R. Canale: "Numerical Methods for Engineers", Mcgraw-Hill, 6th Edition (2010).
3. J. Kilty, A. M. McAllister: "Mathematical Modeling and Applied Calculus", 1st Ed. Oxford University Press (2018).
4. U. Kockelkorn: "Statistik für Anwender", 1st Ed. Springer (2012), s. chapters 7 - 8.
5. Robert H. Shumway, David S. Stoffer: "Time Series Analysis and Its Applications with R Examples", Springer (2011).
6. Gareth James, Daniela Witten, Trevor Hastie, Robert Tibshirani, Jonathan Taylor: "An Introduction to Statistical Learning with Applications in Python", Springer (2023).
7. Cornelis W. Oosterlee, Lech A. Grzelak: "Mathematical Modeling and Computation in Finance with Exercises and Python and MATLAB Computer Codes", World Scientific (2020).
8. Richard Szeliski: "Computer Vision - Algorithms and Applications", Springer (2022).
9. Anthony Scopatz, Kathryn D. Huff: "Effective Computation in Physics - Field Guide to Research with Python", O'Reilly Media (2015).
10. Alex Gezerlis: "Numerical Methods in Physics with Python", Cambridge University Press (2020).
11. Gary Hutson, Matt Jackson: "Graph Data Modeling in Python. A practical guide", Packt-Publishing (2023).
12. Hagen Kleinert: "Path Integrals in Quantum Mechanics, Statistics, Polymer Physics, and Financial Markets", 5th Edition, World Scientific Publishing Company (2009).
13. Peter Richmond, Jurgen Mimkes, Stefan Hutzler: "Econophysics and Physical Economics", Oxford University Press (2013).

14. A. Coryn , L. Bailer Jones: "Practical Bayesian Inference A Primer for Physical Scientists", Cambridge University Press (2017).
15. Avram Sidi: "Practical Extrapolation Methods - Theory and Applications", Cambridge university Press (2003).
16. Volker Ziemann: "Physics and Finance", Springer (2021).
17. Zhi-Hua Zhou: "Ensemble methods, foundations and algorithms", CRC Press (2012).
18. B. S. Everitt, et al.: "Cluster analysis", Wiley (2011).
19. Lior Rokach, Oded Maimon: "Data Mining With Decision Trees - Theory and Applications", World Scientific (2015).
20. Bernhard Schölkopf, Alexander J. Smola: "Learning with kernels - support vector machines, regularization, optimization and beyond", MIT Press (2009).
21. Johan A. K. Suykens: "Regularization, Optimization, Kernels, and Support Vector Machines", CRC Press (2014).
22. Sarah Depaoli: "Bayesian Structural Equation Modeling", Guilford Press (2021).
23. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
24. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
25. Rex B. Kline: "Principles and Practice of Structural Equation Modeling", Guilford Press (2023).
26. Ekaterina Kochmar: "Getting Started with Natural Language Processing", Manning (2022).
27. Jakub Langr, Vladimir Bok: "GANs in Action", Computer Vision Lead at Founders Factory (2019).
28. David Foster: "Generative Deep Learning", O'Reilly(2023).
29. Rowel Atienza: "Advanced Deep Learning with Keras: Applying GANs and other new deep learning algorithms to the real world", Packt Publishing (2018).
30. Josh Kalin: "Generative Adversarial Networks Cookbook", Packt Publishing (2018).
31. Thomas Haslwanter: „Hands-on Signal Analysis with Python: An Introduction“, Springer (2021).
32. Jose Unpingco: „Python for Signal Processing“, Springer (2023).
33. R. K. Burdick, C. M. Borror, D. C. Montgomery: "Design and Analysis of Gauge R&R Studies", 1st Ed. SIAM (2005); S. H. Derakhshan , C. V. Deutsch: "Numerical Integration of Bivariate Gaussian Distribution", Paper 405, CCG Anual Report 13 (2011).
34. C. Paar, J. Pelzl: "Understanding Cryptography", Springer (2010); H. Delfs, H. Knebl: "Introduction to Cryptography", 3rd Ed. Springer (2015); J. Katz, Y. lindell: "Introduction to Modern Cryptography", 2nd Ed, CRC Press (2015); O. Goldreich: "Foundations of Cryptography", Cambridge University Press (2008); J. P. Aumasson: "Serious Cryptography", no starch press (2018).